# Assignment 1
# Git & GitHub Fundamentals

Session 1 — Group D — Individual Lab Work

---

**Full Name:** Sara Benslimane

**Student ID:** 161206

**Course Code & Section:** FA26-CSC332403 — Software Engineering I

**GitHub Username:** sarabenslimane14

**Repository:** https://github.com/sarabenslimane14/git-session1-sarabenslimane14

This report summarizes the Git workflow completed for the Session 1 assignment and includes screenshots of important states and command results.

## 1. Repository Setup

I used the provided starter project and initialized it as a local Git repository, with `main` as the primary branch. I created a `.gitignore` file to ignore `local.env` and normal `.log` files. The assignment requires `important.log` to remain tracked/visible to Git, and it appears in the repository file list.

## 2. README & Commit History

The README includes my name, GitHub username, project description, technologies, a code example, a useful link, and the provided project banner. I used meaningful commit messages to record changes across multiple files. The history includes commits for the README and ignore rules, homepage introduction, application script, styling, important.log, the About feature, and the project banner. The repository is published on GitHub.

## 3. Selective Staging

I modified `index.html` and `style.css`, then staged only `index.html` using `git add index.html`. The status output showed the HTML file under “Changes to be committed” while the CSS file remained under “Changes not staged for commit.” I inspected the staged patch using `git diff --cached`, committed the HTML change, and committed the CSS change separately.

## 4. Branch & Merge

I created an `about` branch for the About feature and committed the feature on that branch. I returned to `main` and ran `git merge about`. Git completed a fast-forward merge, bringing the About section into `main`. I pushed both branches to GitHub.

## 5. Remote Changes

A **commit** records a snapshot in the local repository. A **push** sends local commits to the remote repository. A **fetch** downloads remote updates without integrating them into the current local branch. A **pull** fetches and integrates remote changes into the local branch.

I edited README.md directly on GitHub and committed the change remotely. After `git fetch origin`, `git status -sb` showed that local `main` was behind `origin/main` by one commit. The command `git log --oneline main..origin/main` identified the remote README update. I then ran `git pull origin main`, which fast-forwarded the local branch. The final status showed that the branch was up to date and the working tree was clean.

## 6. Fair Use of AI

I used ChatGPT as a learning and troubleshooting aid during this lab. I asked for explanations of Git commands, help interpreting terminal output, and guidance on staging, branching, merging, fetching, and pulling. I treated the suggestions as steps to test, not as proof that a command succeeded. I verified results in my own terminal using commands such as `git status`, `git diff --cached`, `git branch`, and `git log --oneline --graph --all`. When I mistyped commands and the terminal returned errors, I corrected them and checked the subsequent output.

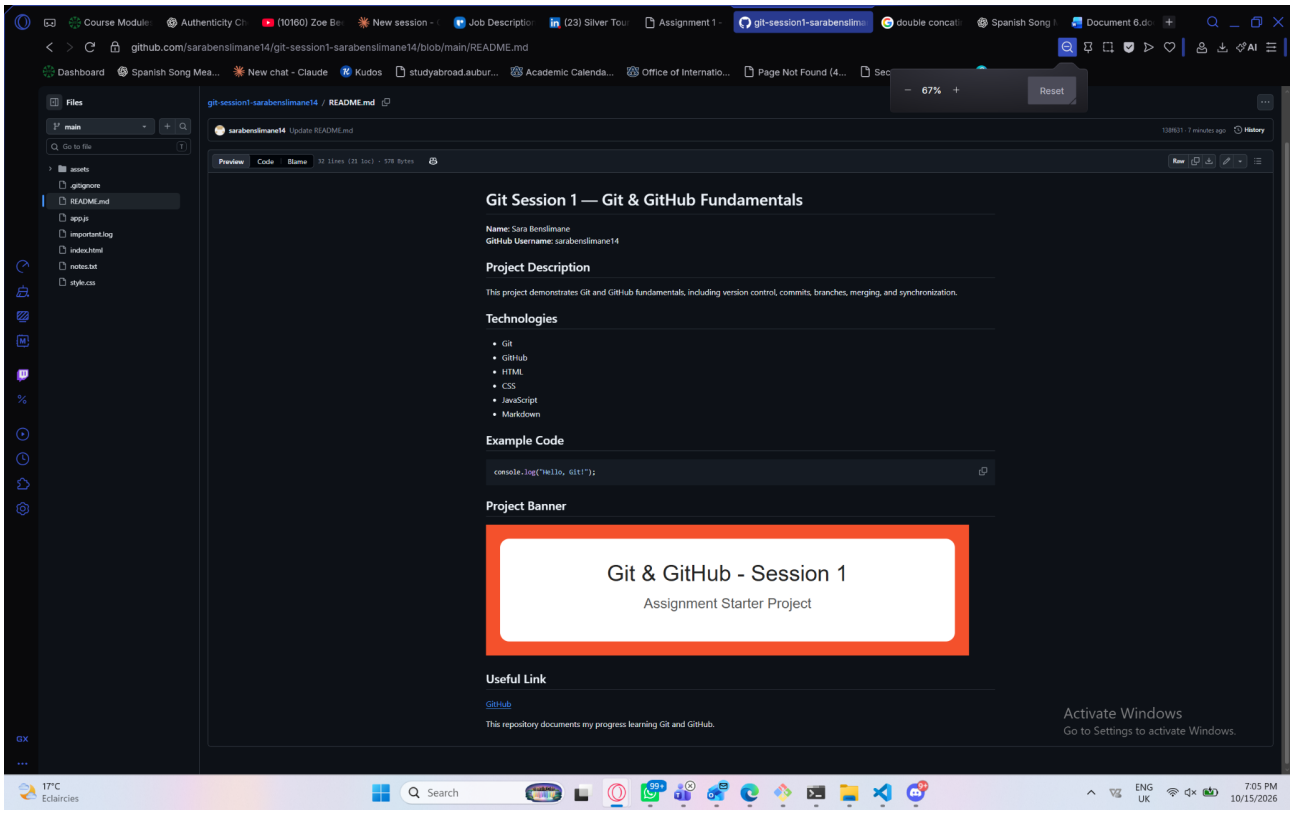
## 7. Sources & Reflection

**Sources:** Assignment 1 and Report Template — Group D (course handout); Git documentation at https://git-scm.com/docs; ChatGPT for explanations and troubleshooting support.

The hardest concept was distinguishing the working tree, staging area, local commits, and remote changes. If Git behaves unexpectedly, I would first check the current branch and working-tree state with `git status`, then inspect changes with `git diff` or `git diff --cached` before trying another command.

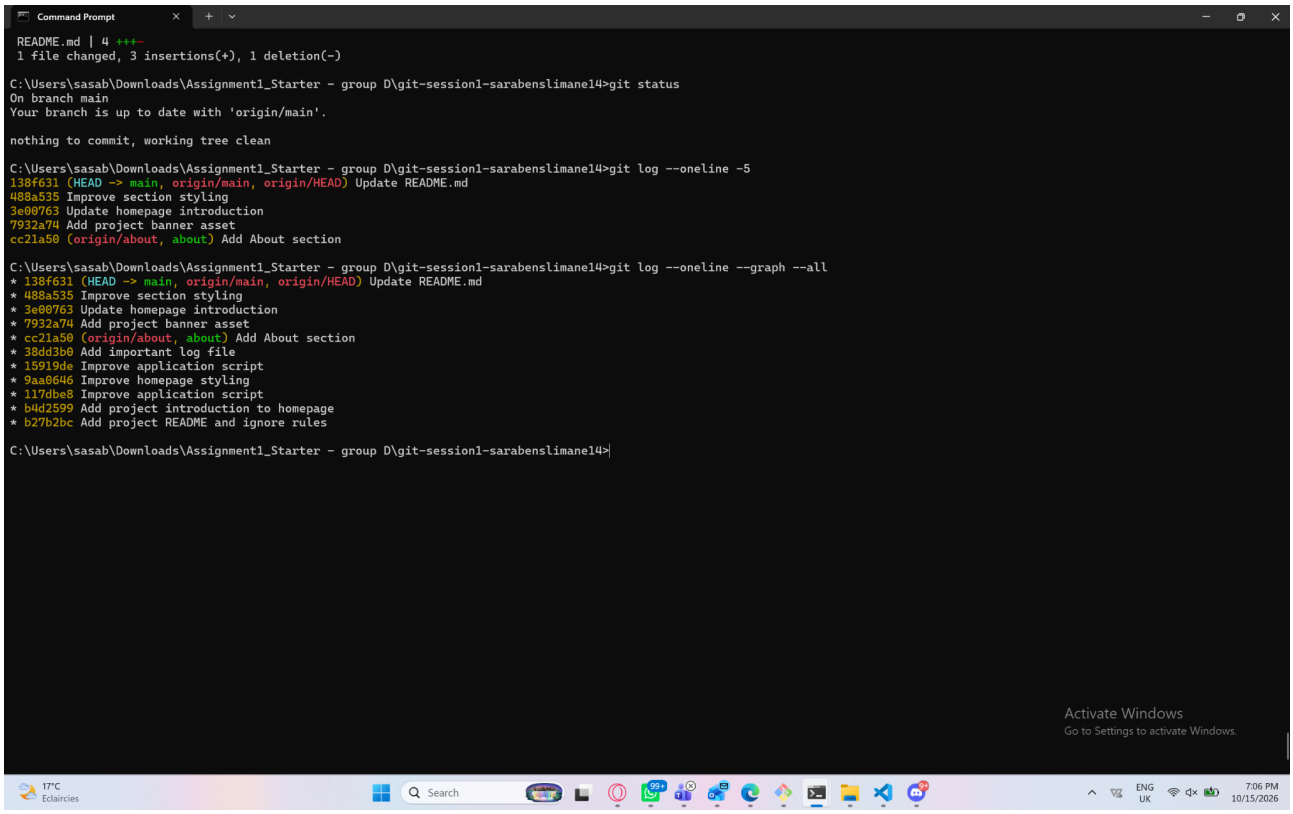
**Submission reminder:** Commit this PDF inside the repository and submit the repository URL in the shared Excel sheet.

# Evidence A — GitHub repository and rendered README



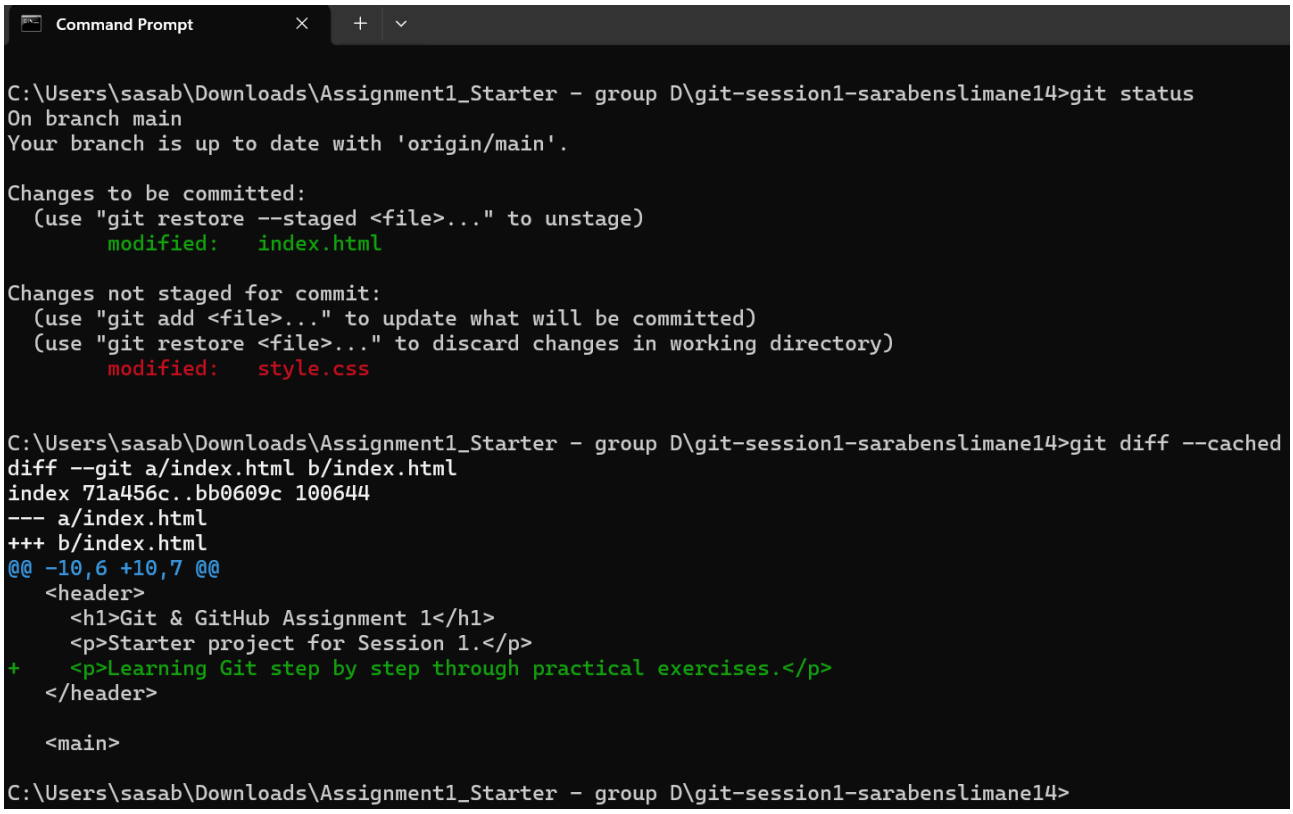

The full-screen GitHub screenshot shows the README content, technologies, code example, project banner, useful link, and repository file list.

## Evidence B — Commit history and branch structure



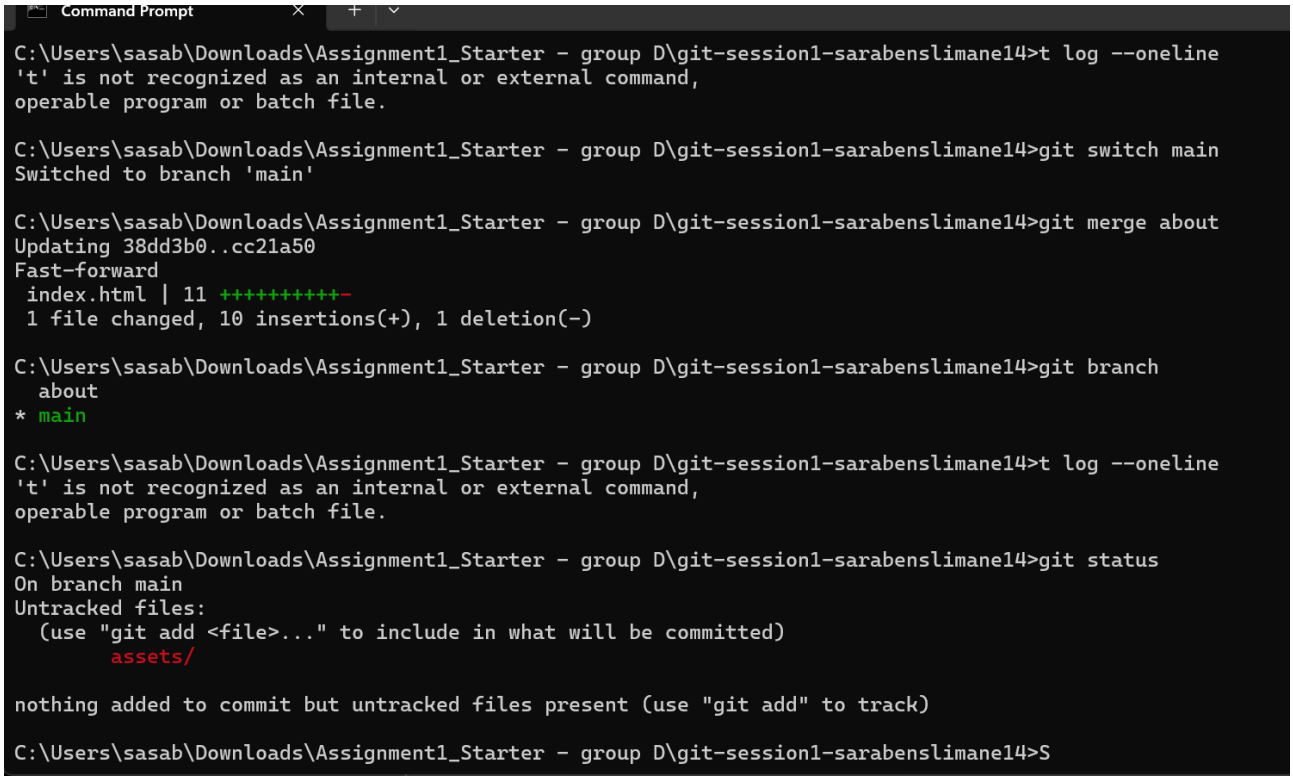

The full-screen terminal screenshot shows the commit graph, the main branch, the about branch, and the commit history.

## Evidence C — Selective staging



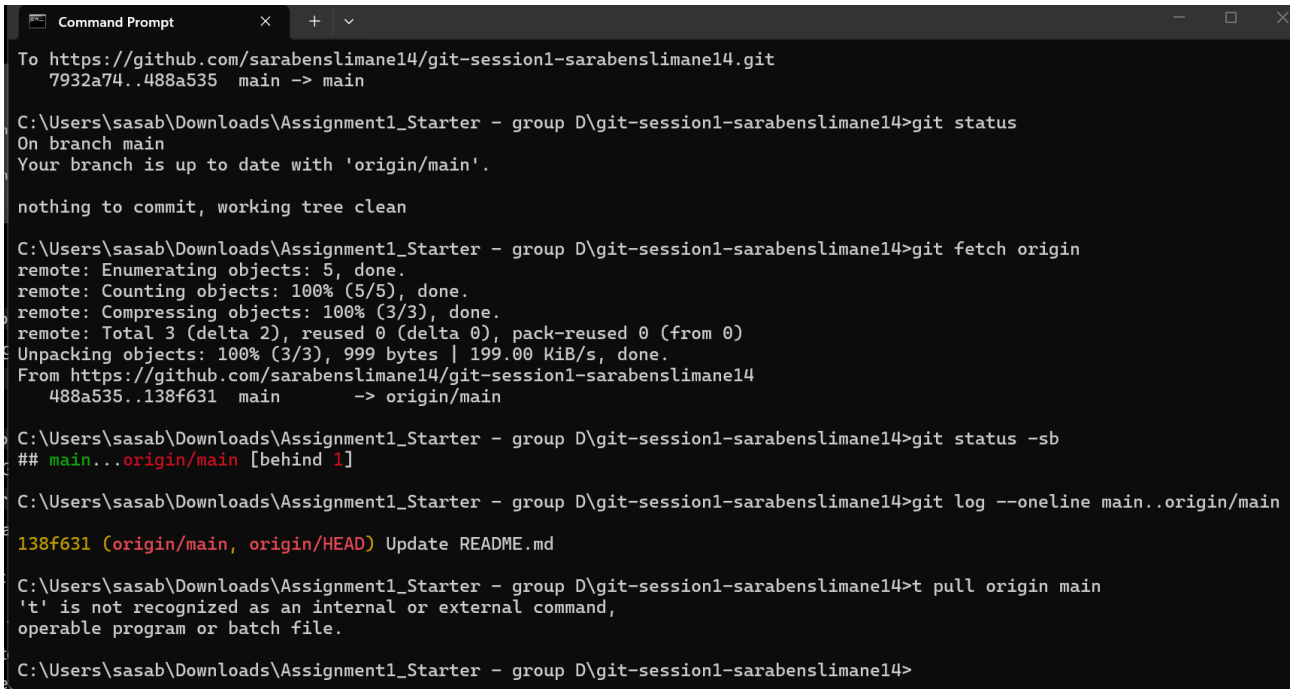

The terminal evidence shows index.html staged while style.css remained unstaged, with the staged HTML diff visible.

## Evidence D — Branch and merge



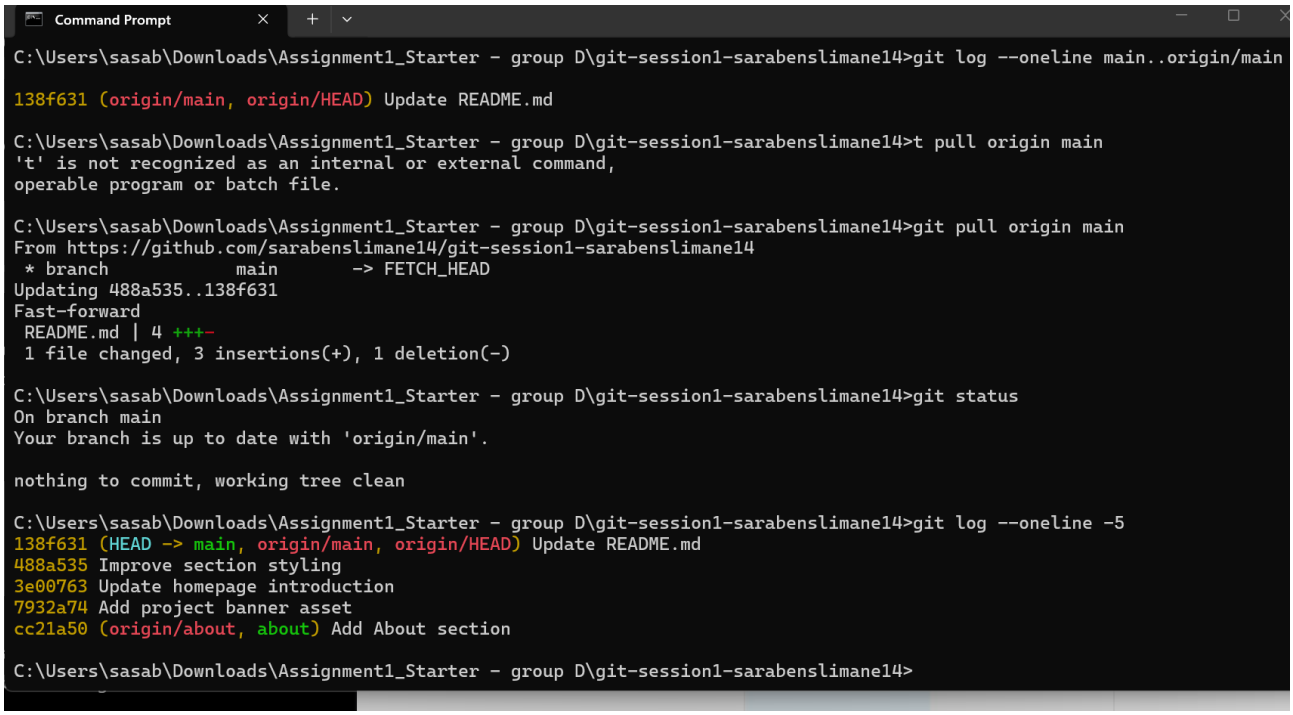

The terminal output shows switching to main and completing a fast-forward merge of the about branch.

## Evidence E — Fetch: remote is ahead



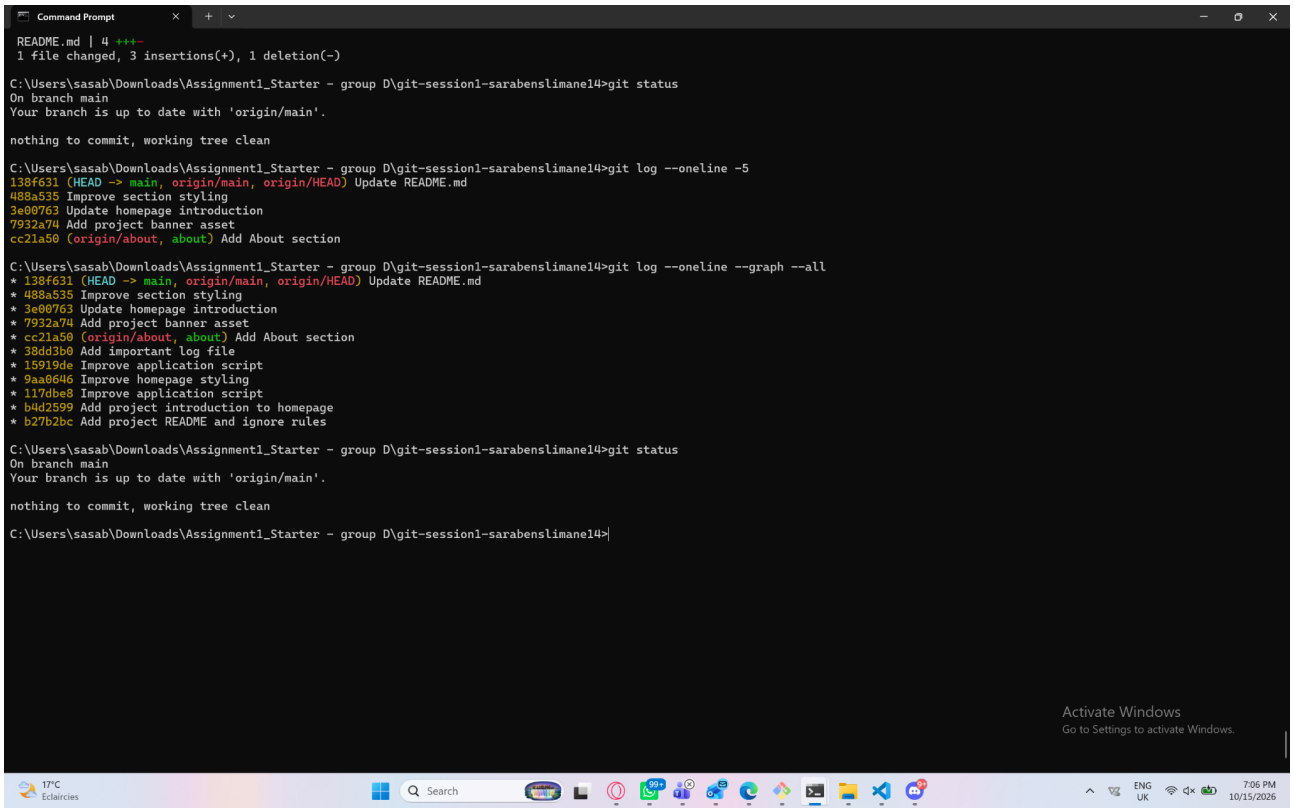

After fetching, the local main branch is shown as behind origin/main by one commit, and the remote-only log identifies the README update.

## Evidence F — Pull and clean final state




The pull fast-forwards the local README. The terminal then reports that main is up to date with origin/main and the working tree is clean.

## Evidence G — Latest full-screen status and history




The latest full-screen terminal screenshot confirms the commit history and the final clean working tree.